



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Writing the Grammar

CMSC 124 Activity 4

Prepared by: Rene Andre Bedonia Jocsing

Published: September 07, 2026

Deadline: September 07, 2026

A token stream tells us what pieces the scanner found. A grammar tells us how those pieces may fit together. Today you'll grow one assignment from a start symbol, turn that history into a tree, and then repair a grammar that permits two incompatible readings.

The grammar here is invented for the activity. It's written to be read. Every rule it depends on is on this page. The notation is the one Laboratory Activity 2 uses. That laboratory is released on September 20. It asks you to write a grammar for your own language and then implement it, so the levels you settle on there turn into parser functions.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 15 points. Nine points reward constructions, and six reward explanations.

The Small Language

A formal grammar describes a **language**, a set of permitted sentences. A **sentence** is a sequence of terminals obtained by starting at the start symbol and applying productions until no nonterminal remains. A **sentential form** is an intermediate sequence that may still contain nonterminals.

A **terminal** is a symbol that can appear in a sentence, such as `id`, `+`, or `=`. A **nonterminal** identifies a category that must still be expanded, such as `<term>`. A **production** states one permitted replacement. The **start symbol** is the nonterminal where generation begins.

This grammar uses Extended Backus-Naur Form (EBNF) to write a context-free grammar (CFG). EBNF adds readable shorthand to BNF without adding descriptive power. A CFG generates a context-free language. Regular languages form a narrower class commonly used for lexical structure, while CFGs can describe nested syntactic structure.

Braces mean zero or more repetitions. A vertical bar means a choice.

```
1 <assign> ::= id = <term>
2 <term>   ::= <factor> { + <factor> }
3 <factor> ::= <primary> { * <primary> }
4 <primary> ::= id | ( <term> )
```

Read `<term> ::= <factor> { + <factor> }` as "term produces a factor followed by zero or more plus-factor pairs." Read `|` as "or."

These level labels are a convention. The grammar above is the one we'll use for the rest of the course, with Sebesta and Wirth putting multiplication in the rule they call `<term>`, one level up from where it sits here. Nystrom, this course, and Laboratory Activity 2 put addition there instead. Precedence comes

from which operator sits lower in the tree. A nonterminal's label decides none of it. So when you meet somebody else's grammar, read the productions and work the levels out from them.

The last production is where you get to overrule the levels. Writing $(b + c) * d$ pushes the addition down into a primary, so it finishes as a subtree before the multiplication combines anything. Activity 7 writes the parser function that consumes those parentheses.

A **derivation** is the sequence of production applications that grows a sentence. In a **leftmost derivation**, always expand the leftmost remaining nonterminal. A **parse tree** records the hierarchical result of those production choices without recording the order in which the derivation applied them. Its root is the start symbol, its internal nodes are nonterminals, and its leaves spell the terminal sentence from left to right.

The Same Rule, Drawn

Wirth published PL/0's grammar as pictures, and plenty of language reports still do. In a **syntax diagram**, you enter at the left edge and follow arrows to the right edge. The circles marked *start* and *end* are that entry and that exit. A rounded box is a terminal, which you write down as you pass through it. A square box is a nonterminal, so you leave this diagram, run the one it points at, and come back where you left.

One rule drawn as a syntax diagram

Rounded boxes are terminals you write down as you pass them. Square boxes send you off to another rule.

Follow every path through term

The straight path produces one factor. The return path adds a plus sign and another factor, and you may take it as many times as you like.

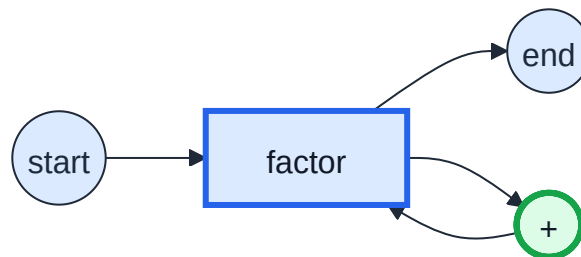


Figure 1: Follow every path through term

Two paths leave the *factor* box. Going straight to the *end* gives the sentence $\langle \text{factor} \rangle$ on its own. Taking the return path instead adds $+ \langle \text{factor} \rangle$ and drops you back at the same fork, so you may collect as many plus-factor pairs as you want. Zero or more trips around that return path is what the braces say:

```
1 <term> ::= <factor> { + <factor> }
```

The conversion is that mechanical. Each path from the left edge to the right edge becomes one alternative on the right of $::=$. Each return path becomes one brace pair. Language reports use both notations, so you'll meet the picture as often as the productions.

Worked Example for Checkpoint 1

Suggested time: 10 minutes, including the worked example.

The rule $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ + \langle \text{factor} \rangle \}$ can produce one factor because the braces may repeat zero times. Repeating the contents twice produces $\langle \text{factor} \rangle + \langle \text{factor} \rangle + \langle \text{factor} \rangle$. The braces and plus sign are grammar notation during expansion. Only the generated plus signs remain terminals in the sentence.

Leaving the braces in the final sentence treats EBNF notation as tokens. Braces are metasympols used to describe repetition, so they aren't tokens in this language.

Checkpoint 1: Read the Rules (3 Points)

Use this grammar for both questions:

```

1  <assign>    ::= id = <term>
2  <term>      ::= <factor> { + <factor> }
3  <factor>    ::= <primary> { * <primary> }
4  <primary>   ::= id | ( <term> )

```

1. (Construction: 2 points) List the four nonterminals, the terminals, and the start symbol. Expand the EBNF braces in $\langle \text{term} \rangle$ just far enough to generate three factors joined by two plus signs.

2. (Explanation: 1 point) Explain why the braces allow both one factor and three factors without adding a second $\langle \text{term} \rangle$ production.

Worked Example for Checkpoint 2

Suggested time: 25 minutes, including the worked example.

Generate $\text{id} = \text{id} * \text{id}$. The derivation expands one nonterminal per step, and unrolling a brace pair counts as a step of its own. Every intermediate form remains visible:

```

1  <assign>
2  => id = <term>
3  => id = <factor> { + <factor> }
4  => id = <factor>
5  => id = <primary> { * <primary> }
6  => id = <primary> * <primary>
7  => id = id * <primary>
8  => id = id * id

```

Steps three and five are where the braces get settled. The $\{ + \langle \text{factor} \rangle \}$ pair repeats zero times, so it disappears and no plus sign survives into the sentence. The $\{ * \langle \text{primary} \rangle \}$ pair repeats once, so one $*$ and one more $\langle \text{primary} \rangle$ join the sentential form. That repetition is where the multiplication in the target came from.

The derivation always expands the leftmost remaining nonterminal. The tree below records the same choices without recording their time order. Its leaves reproduce $\text{id} = \text{id} * \text{id}$, while the $\langle \text{factor} \rangle$ node owns the complete multiplication. That ownership is the grammar's precedence rule.

Putting $*$ directly under $\langle \text{term} \rangle$ skips $\langle \text{factor} \rangle$. That tree produces the right characters but isn't licensed by the grammar. Every parent-child relationship needs a production that permits it.

The three figures below are one tree drawn three times. Each adds the layer the next production supplies. The leaves stay in the order $\text{id} = \text{id} * \text{id}$ throughout.

Grow a parse tree from the grammar

Each step expands one layer while preserving the terminal order $\text{id} = \text{id} * \text{id}$.

Expand the assignment

The start production supplies the target, equals sign, and term slot.

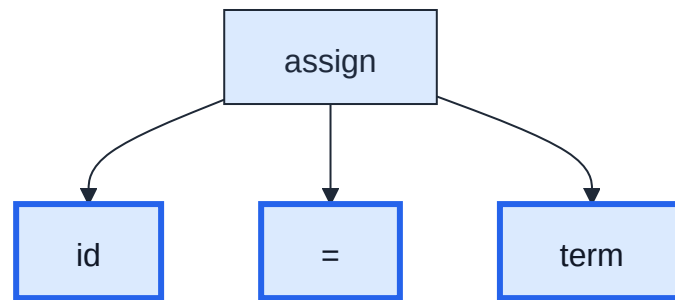


Figure 2: Expand the assignment

Give multiplication its own factor

The term contains one factor. Repetition inside the factor supplies the multiplication and the second primary.

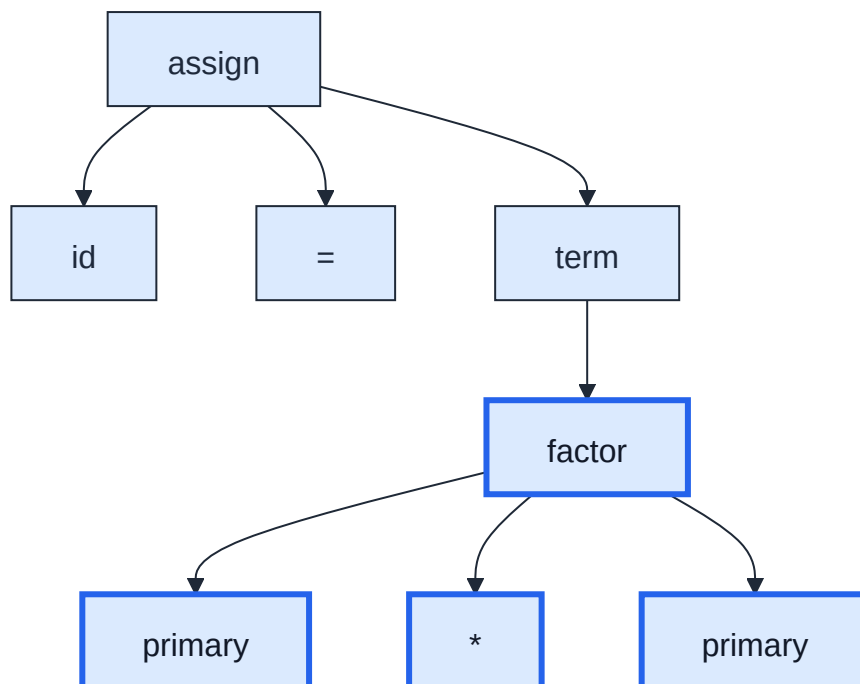


Figure 3: Give multiplication its own factor

Finish at terminal leaves

Each primary chooses `id`, so reading the leaves from left to right reproduces the target sentence.

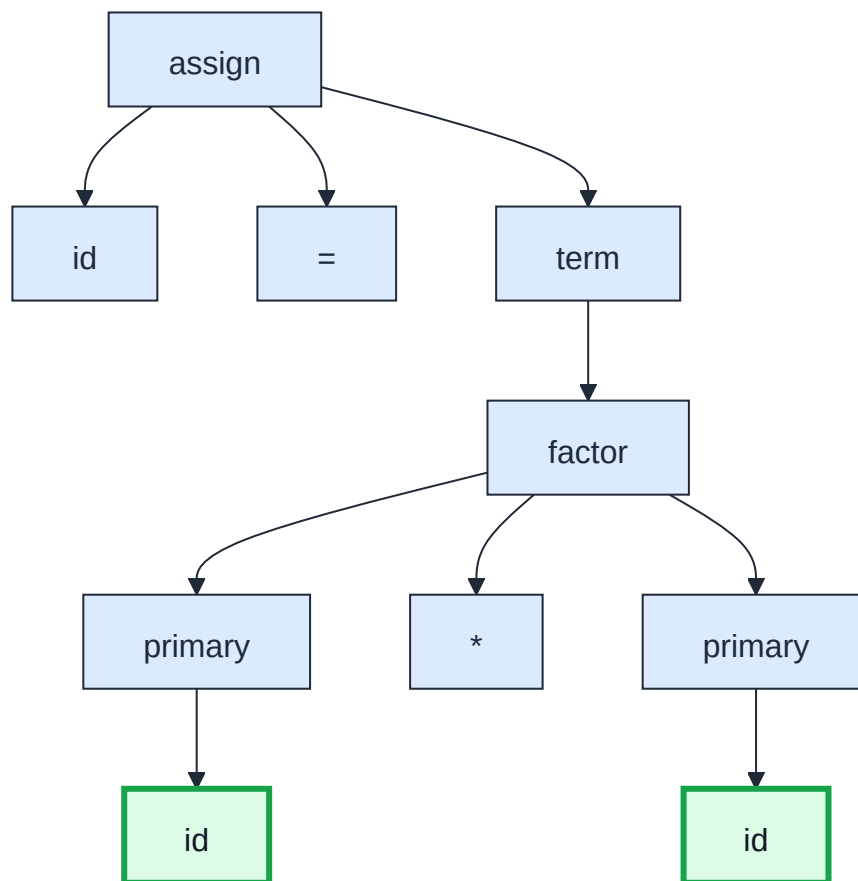


Figure 4: Finish at terminal leaves

A pair of parentheses makes a **grouping** explicit. Compare $B + (C * D)$ with $(B + C) * D$. In the first expression, multiplication **binds more tightly** than addition, so it forms a subtree before addition combines its operands. That ordering is **precedence**. In $A + B + C$, either $(A + B) + C$ or $A + (B + C)$ respects the same precedence. The choice between those equal-precedence groupings is **associativity**.

The EBNF repetition supplies a sequence of operator-operand pairs. It doesn't decide how a parser builds the tree. A **fold** builds one result from a sequence by carrying an accumulated result through it one item at a time. A **left fold** processes those items from left to right. Here the parser starts with the first factor as its accumulated tree and wraps that tree with each next pair:

```

1  start: A
2  fold + B: (A + B)
3  fold + C: ((A + B) + C)

```

The words `start` and `fold` label the action on each line. They aren't grammar terminals. `start: A` puts the first factor, `A`, in the accumulated tree. `fold + B` reads the next pair, the operator `+` and the factor `B`, then combines them with that tree to produce $(A + B)$. That result becomes the new accumulated tree. The last line repeats the action with `+` `C`, producing $((A + B) + C)$. The nested parentheses show that the parser groups $A + B$ first and adds `C` afterward.

Checkpoint 2: Derive and Draw (6 Points)**Grow the Running Sentence**

Use the activity grammar below to derive the target token sentence:

```

1 <assign> ::= id = <term>
2 <term>   ::= <factor> { + <factor> }
3 <factor> ::= <primary> { * <primary> }
4 <primary> ::= id | ( <term> )

```

The target is:

```

1 id = id + id * id

```

Use A, B, C, and D only as annotations beneath the four `id` leaves. They aren't extra terminals in the grammar.

1. (Construction: 4 points) Write a complete leftmost derivation from `<assign>` to the target sentence. Show every intermediate sentential form, including the steps that unroll a brace pair. Then draw its parse tree and annotate the leaves so they read `A = B + C * D`.

2. (Explanation: 2 points) Circle the subtree that combines C and D. Explain how the separate `<term>` and `<factor>` levels make `*` bind more tightly than `+`. Then explain how a parser can fold the repeated plus-factor pairs from left to right. EBNF repetition alone doesn't choose an associativity direction.

When One Sentence Gets Two Trees

A grammar is **ambiguous** when at least one sentence has two distinct parse trees. Those trees have the same terminal leaves in the same order, but their internal nodes group the leaves differently. The grouping can change the meaning. For `id + id * id`, the two readings are `id + (id * id)` and `(id + id) * id`. This is a precedence ambiguity because two different operators compete. For `id + id + id`, the two readings differ only in which addition groups first. That's an associativity ambiguity.

Consider this flatter grammar:

```

1 <term> ::= <term> + <term>
2       | <term> * <term>
3       | id

```

It can generate `id + id * id`, but it doesn't say which operation becomes the lower subtree and therefore groups first. The characters carry neither precedence nor associativity. A grammar, parser rule, or separate language rule must encode both decisions.

The two figures below are that one sentence read two ways. The leaves are identical in both. The roots differ.

One mixed-operator sentence, two trees

The same leaves support two incompatible groupings when the grammar gives neither operator precedence.

Multiply first

Addition is the root, so the tree represents `id + (id * id)`.

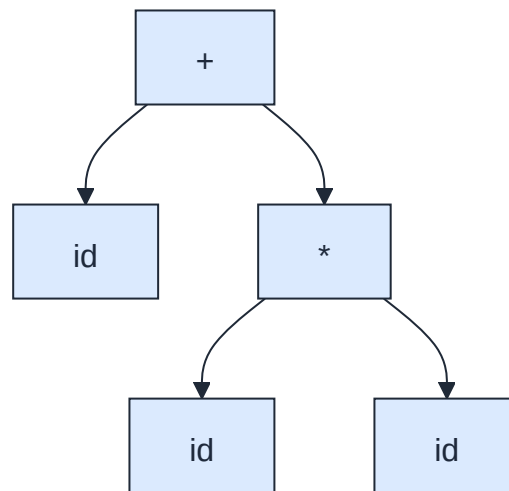


Figure 5: Multiply first

Add first

Multiplication is the root, so the tree represents $(id + id) * id$.

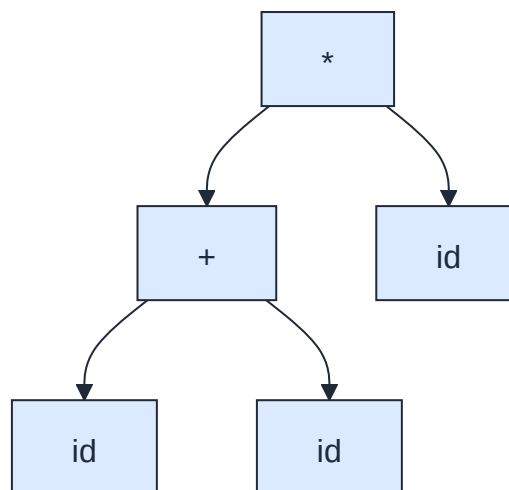


Figure 6: Add first

Worked Example for Checkpoint 3

Suggested time: 25 minutes, including the worked example.

The ambiguity is already visible in $id + id + id$. One tree represents $(id + id) + id$. The other represents $id + (id + id)$. Both have the same leaves, but they group different pairs first.

One addition chain, two trees

A flat recursive rule permits either addition to become the root.

Put the first addition lower

The tree represents $(id + id) + id$, a left-associated reading.

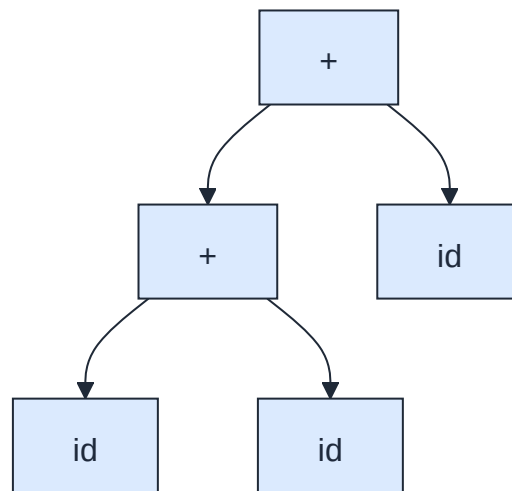


Figure 7: Put the first addition lower

Put the second addition lower

The tree represents $\text{id} + (\text{id} + \text{id})$, a right-associated reading.

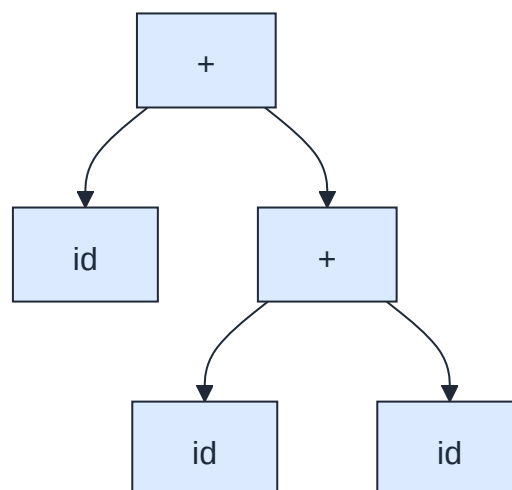


Figure 8: Put the second addition lower

Two Ways to Settle the Grouping

Checkpoint 3 lets you repair the grammar by either of two routes, so here is the second one. You already have the first. Keep the EBNF braces and pair them with the left fold from Checkpoint 2, where the braces supply the sequence and the fold decides it groups from the left.

The other route writes the grouping into the grammar and needs no fold. A rule is **left recursive** when it repeats its own label as the first symbol to the right of $::=$:

```
1 <term> ::= <term> + <factor> | <factor>
```

Read the first alternative from the outside in. A $\langle \text{term} \rangle$ is a smaller $\langle \text{term} \rangle$, then a plus sign, then one $\langle \text{factor} \rangle$. The recursion sits on the left. Every plus sign attaches to a $\langle \text{term} \rangle$ that's already finished, forcing $(A + B) + C$. There's no derivation for $A + (B + C)$ at all.

Move the recursion to the other side and the grouping flips:

```
1 <term> ::= <factor> + <term> | <factor>
```

That rule is **right recursive**. It forces $A + (B + C)$ instead. The only difference between the two grammars is which side of the operator the rule repeats itself on. That single choice is the whole of associativity.

Activity 7 comes back to left recursion, which stalls a recursive-descent parser like the one that activity asks you to write.

Checkpoint 3: Remove the Choice (6 Points)

Use this flat grammar and target together:

```
1 <term> ::= <term> + <term>
2         | <term> * <term>
3         | id
```

Target: $id + id * id$

1. (Construction: 3 points) For $id + id * id$, draw the two distinct parse structures permitted by the flat grammar. Fully parenthesized forms may accompany your trees, but don't replace them.

2. (Explanation: 3 points) Rewrite the flat grammar so every generated sentence gives $*$ higher precedence than $+$. Make both operators associate left. You may use left-recursive BNF, or EBNF paired with an explicit left-fold parser rule. Explain what enforces precedence and what enforces associativity.

Several repaired grammars are possible. Full credit requires all three properties: it still generates the target sentence, the target has only one parse tree, and that tree groups multiplication before addition. End with your repaired grammar beside the explanation that justifies those properties.